

EVO AURA

Alteration Guide for Existing Files

How Codex should alter your current available files into the 41-file structure.

Document Type	PDF 1 of 2 - Alteration only
Target OS	Windows 10 and later only. Do not add Windows 7/8 compatibility changes.
Purpose	Codex-ready instructions for refactoring, packaging, and smooth EXE deployment.
DB Rule	Each computer creates and uses its own local SQLite DB on first run.

Important instruction for Codex: keep the software working while refactoring. Use compatibility imports where needed, test after every file move, and never hardcode user data inside Program Files or the installation folder.

1. Scope and strict rules

This PDF tells Codex how to alter the existing Evo Aura files you already have. It does not exclude any current file shown in your folder screenshot. It also keeps your first-run DB creation behavior for every computer after EXE installation.

Windows target: Windows 10 and later only. Do not change the code for Windows 7 or Windows 8 support. Use modern Windows 10/11 paths, 64-bit Python, and current PyQt6 packaging practices.

Do not rewrite everything in one risky step. Move code page by page, keep wrapper imports during migration, and run the app after each stage.

2. Current files that must not be excluded

Codex must consider every current file below. Some files are source files to refactor, some are assets, and some are build/test artifacts to exclude from the installer.

Current file/folder	Type	Required action during alteration
evoaura_app.py	Python source file	Current main boot/auth/application file. Split into core, auth pages, shell, and main launcher.
loyalty_page.py	Python source file	Keep and move/refactor as pages/customers/loyalty_page.py.
saravana.db	SQLite database file	Use only as sample/test DB or migration source. Do not bundle as default production DB.
__pycache__	Folder	Generated cache. Exclude from Git, Codex refactor, PyInstaller build, and installer.
credit_management_page.py	Python source file	Move/refactor to pages/customers/due_collection_page.py or keep compatibility wrapper.
sidebar.py	Python source file	Move/refactor to core/sidebar.py. Keep NAV_ITEMS aligned with routes.py.
customer_page.py	Python source file	Move/refactor to pages/customers/customer_center_page.py.
low_stock_page.py	Python source file	Move/refactor to pages/inventory/reorder_control_page.py.
purchase_orders_page.py	Python source file	Move/refactor to pages/inventory/purchase_invoices_page.py.
dashboard.py	Python source file	Move/refactor to pages/dashboard_page.py. Class should be DashboardPage.
supplier_page.py	Python source file	Move/refactor to pages/inventory/supplier_page.py.
product_page.py	Python source file	Move/refactor to pages/inventory/product_page.py.
.git	Folder	Keep for source control only. Exclude from final build and installer.
app_branding_qt5.py	Python source file	Old/alternate Qt5 branding helper. Do not use if final software is PyQt6. Archive or convert only if needed.
app_branding.py	Python source file	Move/refactor to core/app_branding.py. Use this for PyQt6 app icon/branding.
input_behavior_qt5.py	Python source file	Old/alternate Qt5 input helper. Do not use if final software is PyQt6. Archive or convert only if needed.
input_behavior.py	Python source file	Move/refactor to core/input_behavior.py. Apply global input guard once at app start.
invoice.pdf	PDF file	Sample invoice/template only. Move to assets/templates if needed. Do not overwrite customer invoices.
icons	Folder	Asset folder. Include in PyInstaller spec and installer package.

3. Required target folder structure before moving code

Create this structure first. Add `__init__.py` to core and every pages subfolder so imports are stable.

```

evo_aura/
main.py
core/
__init__.py
app_shell.py
sidebar.py
routes.py
theme.py
ui_helpers.py
db.py
migrations.py
auth_service.py
input_behavior.py
app_branding.py
pages/
__init__.py
auth/
sales/
inventory/
customers/
finance/
reports/
settings/
assets/
icons/
templates/
installer/
tests/

```

4. Existing file alteration map

Existing file	Target file	Alteration instruction
evoaura_app.py	main.py plus core/app_shell.py plus pages/auth/*.py plus core/db.py plus core/auth_service.py	Extract DB class to core/db.py. Extract TOTP/password logic to core/auth_service.py. Extract helper widgets to core/ui_helpers.py. Extract boot/auth screens to pages/auth. Keep evoaura_app.py as a temporary wrapper that imports main.main() until all shortcuts are updated.
dashboard.py	pages/dashboard_page.py	Rename class to DashboardPage or expose DashboardPage alias. Keep DB access injected from app_shell, not global.
sidebar.py	core/sidebar.py	Move file. Update imports from sidebar to core.sidebar. Keep NAV_ITEMS and active-key logic. Navigation keys must match core/routes.py.
product_page.py	pages/inventory/product_page.py	Move as ProductPage. Keep existing UI first, then clean imports, DB access, search, pagination, and stock/variant logic.
supplier_page.py	pages/inventory/supplier_page.py	Move as SupplierPage. Connect supplier selection to purchase invoices and supplier ledger.
purchase_orders_page.py	pages/inventory/purchase_invoices_page.py	Move or rename as PurchaseInvoicesPage. If class name remains PurchaseOrdersPage, create alias PurchaseInvoicesPage = PurchaseOrdersPage.
low_stock_page.py	pages/inventory/reorder_control_page.py	Move as ReorderControlPage. Keep low-stock table but rename page title to Reorder Control.
customer_page.py	pages/customers/customer_center_page.py	Move as CustomerCenterPage. Add/keep customer search, ledger, sale history.
credit_management_page.py	pages/customers/due_collection_page.py	Move as DueCollectionPage. Keep credit/due collection logic.
loyalty_page.py	pages/customers/loyalty_page.py	Move as LoyaltyPage. Keep loyalty logic and connect to customer records.
app_branding.py	core/app_branding.py	Use as final PyQt6 branding helper. Must support PyInstaller resource path with sys._MEIPASS.
app_branding_qt5.py	archive/ or remove from runtime	Do not import in PyQt6 app unless converted. Exclude from runtime if not needed.

Existing file	Target file	Alteration instruction
input_behavior.py	core/input_behavior.py	Use as final PyQt6 input guard. Call ensure_global_input_guard(app) once in main.py.
input_behavior_qt5.py	archive/ or remove from runtime	Do not import in PyQt6 app unless converted. Exclude from runtime if not needed.
icons/	assets/icons/	Move/copy icon PNGs. Include in PyInstaller datas. Sidebar icon loader must use resource_path.
invoice.pdf	assets/templates/invoice.pdf if used	Keep only as template/sample. Customer invoices should be generated to LocalAppData/Documents folder, not overwrite this file.
saravana.db	tests/sample_data/saravana.db or backup/	Do not ship as user default DB. Only use for migration testing or demo import.
.git	.git	Source control only. Exclude from PyInstaller and installer.
__pycache__	delete/exclude	Generated cache. Delete before packaging.

5. Route and import alteration rules

After moving files, replace direct imports inside the old main file with route-based lazy imports. This prevents heavy pages like product_page.py from loading before login and improves startup speed.

Use this route registry pattern:

```
# core/routes.py
ROUTES = {
    "home": ("pages.dashboard_page", "DashboardPage"),
    "new_sale": ("pages.sales.new_sale_page", "NewSalePage"),
    "product": ("pages.inventory.product_page", "ProductPage"),
    "suppliers": ("pages.inventory.supplier_page", "SupplierPage"),
    "customer": ("pages.customers.customer_center_page", "CustomerCenterPage"),
}

def load_page(route_key, context):
    module_name, class_name = ROUTES[route_key]
    module = importlib.import_module(module_name)
    page_cls = getattr(module, class_name)
    return page_cls(context=context)
```

Fix sidebar keys that are inconsistent. Current sidebar has keys like pl_summaries and securitys in some versions. Codex must either keep backward-compatible aliases or rename both sidebar and routes to pl_summary and security at the same time.

6. DB path and first-run DB creation in EXE

Your app should create a separate database on each computer. The installer should not create a shared DB in the installation folder. The app should create DB only after StartAuth/AskDB/MasterAuth flow.

Recommended Windows 10/11 local data path:

```
%LOCALAPPDATA%\EvoAura\data\<workspace_name>.db
%LOCALAPPDATA%\EvoAura\logs\evoaura.log
%LOCALAPPDATA%\EvoAura\exports\
%LOCALAPPDATA%\EvoAura\backups\
```

Do not write runtime DB files inside Program Files because normal users may not have write permission and updates may overwrite data.

DB workflow Codex must preserve:

- 1 On app start, check a config file in LocalAppData for the last selected DB path.
- 2 If no DB exists or config is missing, show Ask DB Name page.
- 3 After DB name entry, verify Master TOTP in Master Auth page.

- 4 Create the SQLite DB inside LocalAppData/EvoAura/data.
- 5 Run migrations immediately after DB creation.
- 6 Show Company Setup, then admin Signup, then QR setup, then Login.
- 7 If DB exists but users table has no user, go directly to Signup and QR setup.
- 8 If DB and users exist, show Login and OTP, then App Shell.

7. Migration and database alteration instructions

Move table creation out of `evoaura_app.py` into `core/migrations.py`. Migrations must be idempotent: running them multiple times should not duplicate columns or break existing databases.

Table group	Required tables
Auth/company	company, users, roles, permissions, audit_log
Inventory	products, product_variants, barcodes, stock_movements, stock_adjustments
Suppliers/purchase	suppliers, purchase_invoices, purchase_items, supplier_payments
Sales/POS	sales, sale_items, payments, returns, return_items
Customers/credit	customers, customer_ledger, due_payments, loyalty_points
Finance/reports	expenses, expense_categories, cash_transactions, tax_summary
System	app_settings, backup_history, export_history

Add indexes for search and billing performance: product name, barcode, SKU/item_code, customer phone, sale invoice number, sale date, supplier name, and stock status.

8. Smooth performance and no-lag alteration rules

Codex should apply these changes while refactoring:

- Lazy-load main pages only when the user clicks the sidebar item. Do not instantiate all 23 pages at startup.
- Use `QTableView` with `QAbstractTableModel` for large tables instead of filling huge `QTableWidget` rows repeatedly.
- Use pagination or limit queries for product, supplier, customer, bill view, and report pages.
- Run heavy exports, backups, imports, and report generation in `QThread` or `QRunnable` so the UI does not freeze.
- Cache tinted sidebar icons and logo pixmaps instead of repainting/re-tinting every hover event if lag appears.
- Open one DB connection per operation/context and close safely. Avoid long transactions during UI painting.
- Use debounced search: wait 250-350 ms after typing before querying product/customer data.
- Do not rebuild full pages on every navigation. Reuse page widgets and refresh data only when needed.
- Create indexes during migrations. This matters more than UI polish for fast search.

9. EXE and installer procedure for Windows 10/11

Build on Windows 10 or Windows 11 using 64-bit Python. Do not make special changes for Windows 7/8.

Recommended build flow:

- 1 Create and activate a clean virtual environment.
- 2 Install dependencies: `PyQt6`, `pyotp`, `qrcode[pil]`, `Pillow`, `reportlab/openpyxl` only if used by the app.
- 3 Run the app from source and complete the first-run DB flow successfully.

- 4 Build PyInstaller one-folder first. Test startup, login, sidebar icons, all imported pages, and DB creation.
- 5 After one-folder works, optionally build one-file. One-folder is usually smoother for PyQt apps.
- 6 Create installer with Inno Setup or NSIS. Include dist folder, icons folder, templates, and license/readme if needed.
- 7 Install on a fresh Windows 10/11 PC. Confirm DB is created in LocalAppData and not inside the install folder.

Example PyInstaller command:

```
pyinstaller --noconfirm --clean --windowed --name "EvoAura" ^
--icon assets\icons\app.ico ^
--add-data "assets\icons;assets\icons" ^
--add-data "assets\templates;assets\templates" ^
--hidden-import PyQt6.QtSvg ^
main.py
```

For final packaging, prefer a .spec file so datas, hidden imports, and EXE metadata are stable.

Runtime resource helper required in core/app_branding.py:

```
def resource_path(relative_path):
    import os, sys
    base = getattr(sys, "_MEIPASS", os.path.abspath("."))
    return os.path.join(base, relative_path)
```

10. Installer rule for data safety

The installer can install the app to Program Files or per-user AppData, but the database and exports must always be stored under LocalAppData or a user-selected folder. The uninstaller must not delete customer DBs unless the user clearly selects a separate Remove All Data option.

11. Testing checklist after alteration

Area	Test
Startup	Fresh install opens Ask DB Name. Existing install opens Login. No console window in GUI build.
Auth	Master TOTP, admin signup, QR setup, login, OTP, recovery code all work.
DB	New DB is created per computer in LocalAppData. saravana.db is not overwritten.
Navigation	All 23 sidebar pages open without ImportError. Active sidebar highlight works.
Inventory	Product, supplier, purchase invoice, reorder control pages load and save correctly.
Customer	Customer center, due collection, loyalty pages load and connect to DB.
Sales	New sale, return, bill view flow can save, reprint, and update stock.
Reports	Day book, stock report, GST, P&L pages query without freezing UI.
Build	EXE runs on Windows 10/11 without source files installed. Icons load.

12. Codex task prompt for this PDF

You are editing my Evo Aura PyQt6 billing software. Refactor my current available files into the 41-file structure. Do not exclude any existing file from the alteration plan. Preserve the first-run database creation flow for every computer after EXE installation. Target Windows 10 and later only; do not add Windows 7/8 compatibility. Move runtime DBs, logs, exports, and backups to LocalAppData/EvoAura. Use lazy page loading, stable routes, compatibility wrappers during migration, and make the EXE/installer with PyInstaller plus Inno Setup after source run works.